Cronjob

Schedule recurring SQL in a live DuckDB process.

Install & load

```
INSTALL cronjob FROM community;
LOAD cronjob;
```

01 At a glance

Register SQL with a six-field cron expression, inspect jobs and cancel by ID. The scheduler lives inside the DuckDB process, making it useful for short-lived local automation.

SQL + schedule → Live scheduler → Inspect / cancel

02 Core SQL surface

`cron(sql, schedule)`

Register a query; returns the job ID as VARCHAR.

`cron_jobs()`

Table function listing scheduled jobs and their status.

`cron_delete(job_id)`

Cancel a job using the ID returned by cron.

`cron_job_history()`

Last 100 runs per job: start, finish, success, result (DuckDB 2.0).

`second minute hour day month weekday`

Six fields: seconds come first, unlike five-field system cron.

`0 */5 * * * *`

Every five minutes, at second zero.

`0 0 2 * * *`

Daily at 02:00:00 UTC.

03 Keep in mind

Keep the process alive

The scheduler stops when DuckDB exits. Register jobs again after a restart; this is not a persistent system service or a replacement for an external orchestrator.

04 Register a job

```
CREATE TEMP TABLE scheduled AS
SELECT cron(
  'SELECT 42', '0 */5 * * * *'
) AS job_id;
```

The example retains the actual ID so cancellation remains reliable.

05 Inspect jobs

```
SELECT * FROM cron_jobs();
```

Review schedule and status while the same DuckDB process remains running.

06 Cancel the registered job

```
SELECT cron_delete(job_id) AS removed
FROM scheduled;
```

Cancel the example job when finished; no future execution remains scheduled.

07 In practice

Keep and use the returned ID

Do not assume a job will always be task_0. Store the returned identifier and use it for cancellation. A successful registration does not establish that the scheduled SQL will succeed.

Full docs & examples ↗ Source ↗